

Non-FUNCTIONAL REQUIREMENTS

VERSION 1.0

This template was created to enable departments to more easily develop their project plans. The Department of Technology, Consulting and Planning Division, created this template based on its experiences. The template relies on industry best practices combined with decades of experience on California state information technology projects. The way it was structured is to enable a department to complete the information related to its project without having to write background information related to the discipline. A department may use as much or as little of the template as it wishes.

Template Instructions:

- ***Instructions for completing*** this template – written for the author of the project plan - are encased in [] and the text is ***italicized and bolded***.
- *Examples* are provided as a guideline to the type of sample information presented in each section and the text is *italicized*.
- Boilerplate standard language for each section is written in the document font and may be used or modified, as necessary.
- A department's project specific information goes within the brackets << >>.
- *Informational text is italicized* within square brackets [] for informational purposes to the person who has to create the plan and includes background information, explanation, rationale, etc.

DOCUMENT HISTORY

DOCUMENT REVISION HISTORY			
DATE	DOCUMENT VERSION	REVISION DESCRIPTION	AUTHOR
12/09/2014	1.0	Initial Version	R Peterson

TABLE OF CONTENTS

1. INTRODUCTION	1
2. OVERVIEW	1
3. REFERENCED DOCUMENTS	3
4. NON-FUNCTIONAL REQUIREMENTS	10
4.1. PERFORMANCE	10
4.1.1. USER RESPONSIVENESS	10
4.1.1.1 Normal User Load	11
4.1.1.2 End-of-Month Load	12
4.1.2. BATCH PROCESSING	12
4.2. SECURITY	9
4.2.1. AUDITING	6
4.2.1.1 USER ACTION AUDITING	6
4.2.1.2 DATA MODIFICATION AUDITING	7
4.2.2. AUTHENTICATION & AUTHORIZATION	8
4.2.3 Data Protection	
4.3. COMPATIBILITY	8
4.3.1. CROSS-BROWSER COMPATIBILITY	8
4.3.2. LEGACY INTERFACE COMPATIBILITY	9
4.3.3. ETC. COMPATIBILITY	9
4.4. MAINTAINABILITY	10
4.4.1. ERROR REPORTING	10
4.4.1.1 Application Errors	10
4.4.1.2 Database Errors	10
4.4.1.3 API Errors	10
4.4.1.4 Frontend Errors	10
APPENDIX A. CONSIDERATIONS	A-1

1. INTRODUCTION

Non-functional requirements describe the quality attributes, constraints, and operating conditions of a software system rather than its specific functional behavior. While functional requirements define what the system does, non-functional requirements define how well the system performs its functions and under what conditions it operates.

In the context of the Garage Management System, non-functional requirements identify key constraints and quality characteristics such as system performance, security, usability, reliability, scalability, and maintainability. These requirements ensure that the system is stable, secure, responsive, and suitable for daily administrative and technical operations within a garage environment.

This document specifies the non-functional requirements for the Service Booking & Garage Management System, as elicited from project stakeholders including administrators, technical staff, and the development team. The requirements have been analyzed and refined to reflect the current project scope, which focuses primarily on the Admin System and Technician-related functionalities, while excluding customer-facing features that have not yet been implemented.

The non-functional requirements defined in this document establish clear constraints and quality expectations for system development and deployment. They provide a basis for system evaluation, testing, and future enhancement, ensuring that the Garage Management System meets acceptable standards of performance, security, and usability throughout its lifecycle.

2. OVERVIEW

This section provides a brief description of the system, its purpose, architecture, and operational context.

2.1 SYSTEM PURPOSE

The Garage Service Provider System is a web-based application designed to help garage owners and staff manage daily operations efficiently. It supports administrators and technical staff in handling service requests, inventory, products, categories, user roles, reporting, and batch operations. The system consists of a backend API and an admin dashboard, with no public customer-facing features at this stage.

2.2 TECHNOLOGY STACK

Backend: FastAPI (Python)

Database: PostgreSQL or MySQL (with SQLAlchemy ORM)

Frontend (Admin Dashboard): React/Next.js (accessible at <https://garas-admin.domrey.online/>)

Authentication: JWT (JSON Web Tokens)

Background Processing: Asynchronous task queue for batch jobs and notifications
Deployment: Cloud-based infrastructure supporting horizontal scaling

2.3 KEY USERS AND EXPECTED LOAD

Primary users: Garage administrators, managers, and technical staff (typically 5–20 users per garage)

Normal daily concurrent users: Up to 100

Peak periods (e.g., end-of-month reporting): Up to 300 concurrent users

The system is intended for single-location or small multi-branch garage operations.

2.4 SCOPE OF NON-FUNCTIONAL REQUIREMENTS

These requirements focus on how the system performs its functions (performance, security, compatibility, maintainability) rather than what functions it provides. They ensure the system is fast, secure, easy to use and maintain, and works reliably across supported environments.

3. REFERENCED DOCUMENTS

The following documents were referenced and used in creating this deliverable; they also provide definitions and descriptions of non-functional requirements that are used in this document.

- [1] N. Rahmawati et al., "Online Booking Website for Car Service and Spare Parts Availability," 2023.
- [2] Raman et al., "LINKS Automobile Service Management and Reminder System," 2025.

4. NON-FUNCTIONAL REQUIREMENTS

4.1 PERFORMANCE

The system must be fast, stable, and reliable. It shall give users a smooth experience during normal daily use and during busy periods (like end-of-month reporting).

Performance will be measured mainly by:

- How quickly pages and actions respond
- How many users can work at the same time
- Overall system speed and stability

4.1.1 USER RESPONSIVENESS

The system must react quickly to everything a user does, so the application feels easy and pleasant to use. We measure this by page load times, API response times, and how fast actions complete.

4.1.1.1 NORMAL DAILY USE

During regular everyday work, the system must handle normal activity smoothly without any noticeable slowdown.

- The system must support up to 100 users working at the same time without becoming slow for anyone.
- Web pages must fully load in 2 to 3 seconds on a normal internet connection.
- Common actions (like logging in, clicking menus, saving forms, or viewing data) must finish in 1 second or less.
- Normal API requests from the backend must respond in 1 to 2 seconds.
- The system must stay stable: no errors, no long delays, and no freezing of the screen.

4.1.1.2 End-of-Month Load

During peak usage periods, such as end-of-month reporting and administrative processing, the system shall continue to operate reliably.

- The system shall support up to 300 concurrent users during peak periods.

-
- Page load times shall not exceed 5 seconds under high load conditions.
 - Backend API response times shall not exceed 3 seconds for critical operations.
 - Report generation and batch operations shall not block or degrade real-time user interactions.
 - The system shall prioritize essential user operations to ensure continued usability during peak load.

4.1.2 BATCH PROCESSING

The system must support efficient and reliable batch processing capabilities to handle large-scale data processing, reporting, and maintenance activities without compromising real-time user operations. Batch processing must be designed to operate asynchronously, ensuring system stability, data integrity, and operational consistency during peak and off-peak hours.

4.1.2.1 SCHEDULED BATCH JOBS

The system will run large background tasks (called batch jobs) at planned times when few users are active, so it doesn't slow down the system for people using it.

Monthly Reporting:

The system will create detailed monthly reports about services and operations. It will handle up to 10,000 service records and finish the job within a 2-hour quiet period (from 2:00 AM to 4:00 AM local time).

Inventory Reconciliation:

Every day, the system will update and match inventory records. It will process up to 5,000 product records and complete the task in 15 minutes or less.

Data Retention and Archiving:

Every three months, the system will clean up old data by archiving or deleting service records that are more than 12 months old. It will handle up to 50,000 records and finish within 4 hours

4.1.2.2 BATCH OPERATIONS

The system must allow both regular users and administrators to start batch operations. These operations will include proper checks, tracking, and clear feedback.

Batch Import:

The system lets users import up to 1,000 products or inventory items at once using files like CSV or Excel. The import must finish within 5 minutes, but only after checking that the data follows the rules.

Batch Updates:

The system allows updating prices for up to 500 products in one operation. All changes will be checked first before saving to the database, and the whole process must finish within 2 minutes. If some items have errors, the system will report those errors but still save the correct ones.

4.1.2.3 PROCESS RULES

To keep the system reliable and fast, the following rules apply to all batch operations:

Batch operations must run in the background (asynchronously) so they do not slow down or block normal real-time requests.

If a batch operation fails, the system will automatically try again up to 3 times, waiting longer between each try (exponential backoff).

Every batch operation must save details about what happened, such as start time, end time, progress, and results. This information is used for checking, auditing, and fixing problems.

4.2 SECURITY

4.2.1 AUDITING

4.2.1.1 USER ACTION AUDITING

ID#	<i>NFR-SE C-001</i>	Administrative Action Logging
Scenario:	<i>As a system administrator, I want all administrative actions to be logged with sufficient detail, So that we can ensure accountability and traceability of sensitive operations.</i>	
Requirement:	<i>All administrative actions shall be logged with sufficient detail for accountability</i>	
Constraints:	<i>Format Constraint: Timestamp must follow ISO 8601 with timezone.</i>	

ID#	NFR-SE C-001	Administrative Action Logging
		Access Constraint: Logs must only be accessible to system administrators (enforced by role-based access control). Retention Constraint: Logs must be stored for at least 90 days and then securely purged. Security Constraint: Logs must not expose sensitive data beyond what is required for accountability. Performance Constraint: Logging should not degrade system performance beyond acceptable thresholds. Compliance Constraint: Must adhere to organizational security policies and any applicable regulations (e.g., GDPR if personal data is involved).
Verification Method:		Automated Testing • Unit tests to ensure logging occurs for each sensitive operation. • Integration tests to validate log entries contain all required fields (timestamp, user ID, action type, etc.). • Negative tests to confirm failure cases are logged correctly. Manual Log Review • Inspect sample logs to verify completeness and accuracy. • Check retention policy by reviewing logs older than 90 days. Access Control Validation • Attempt to access logs with non-admin accounts to confirm access is restricted. • Verify role-based permissions in the system. Performance Monitoring • Measure system performance during logging operations to ensure no significant degradation. Compliance Audit

ID#	NFR-SE C-001	Administrative Action Logging
		Validate adherence to ISO 8601 timestamp format and security policies.
	Notes:	Logging Middleware: Implement a centralized logging mechanism to ensure consistency across all admin actions. Schema Standardization: Use a structured format (e.g., JSON) for logs to include all required fields. Security Considerations: Avoid logging sensitive data such as passwords or tokens; mask or hash where necessary. Retention Enforcement: Implement automated jobs to purge logs after 90 days securely. Performance Impact: Use asynchronous logging or queue-based systems to prevent performance degradation.

ID#	NFR-SE C-002	Authentication Event Logging
	Scenario:	<i>As a security administrator</i> <i>I want all authentication attempts to be logged</i> <i>So that we can monitor for suspicious activity and ensure account security</i>
	Requirement:	<i>All authentication attempts shall be logged for security monitoring</i>
	Constraints:	Format Constraint: Timestamp must follow ISO 8601 with timezone Access Constraint: Logs accessible only to system administrators Retention Constraint: Logs retained for at least 90 days

ID#	NFR-SE C-002	Authentication Event Logging
		Security Constraint: Logs must not expose sensitive data (e.g., passwords) Performance Constraint: Logging should not degrade authentication performance Compliance Constraint: Must adhere to organizational security policies and regulations
Verification Method:		<i>Automated Tests:</i> • Unit tests for logging on each authentication event • Integration tests for alerting logic (e.g., >5 failed attempts in 15 min) <i>Manual Review:</i> • Inspect logs for completeness and accuracy <i>Access Control Validation:</i> • Confirm logs are restricted to admins <i>Performance Monitoring:</i> • Ensure logging does not slow down login flow <i>Compliance Audit:</i> • Validate adherence to security policies
Notes:		Implement centralized authentication logging using a secure logging service. Use structured log format (e.g., JSON) for easy parsing and monitoring. Consider real-time alerting via SIEM or monitoring tools. Avoid logging sensitive credentials; only log metadata.

ID#	<i>NFR-SE C-002</i>	Authentication Event Logging
		<i>Ensure rate-limiting and account lockout policies complement logging for security.</i>

4.2.1.2 DATA MODIFICATION AUDITING

ID#	NFR-SE C-003	Critical Data Change Tracking
Scenario:	<i>As a system administrator</i> <i>I want all changes to critical business data to be traceable</i> <i>So that we can maintain accountability and auditability for sensitive operations</i>	
Requirement:	<i>Changes to critical business data shall be traceable.</i>	
Constraints:	Format Constraint: Timestamp must follow ISO 8601 with timezone Access Constraint: Audit logs accessible only to authorized administrators Security Constraint: Audit logs must not expose sensitive data beyond what is required Performance Constraint: Logging should not degrade transaction performance Compliance Constraint: Must adhere to organizational security and regulatory requirements	
Verification Method:	Automated Tests: • Unit tests for audit logging on each tracked entity • Integration tests for audit trail completeness Manual Review: • Inspect audit logs for accuracy and retention Access Control Validation: • Confirm only admins can view audit history Performance Monitoring: • Ensure minimal impact on data update operations	

ID#	NFR-SE C-003	Critical Data Change Tracking
		Compliance Audit: Validate adherence to retention and security policies
	Notes:	Use immutable storage or append-only tables for audit records. Consider versioning for tracked entities to allow rollback or historical comparison. Implement change reason as optional but encourage usage for accountability. For Phase 2, integrate booking status changes into the same audit mechanism. Ensure UI for administrators to view audit history easily. If using database triggers, ensure they are transaction-safe and do not break atomicity.

4.2.2 AUTHENTICATION & AUTHORIZATION

ID#	NFR-SE C-004	Password Storage
	Scenario:	<i>As a security administrator</i> <i>I want user passwords to be stored securely using industry-standard cryptographic hashing</i> <i>So that user credentials are protected against compromises</i>
	Requirement:	<i>User passwords shall be stored using industry-standard cryptographic hashing.</i>
	Constraints:	Security Constraint: No plaintext passwords in storage or logs

ID#	NFR-SE C-004	Password Storage
		Compliance Constraint: Must adhere to organizational security policies and applicable regulations Performance Constraint: Hashing should not degrade authentication performance beyond acceptable limits
Verification Method:		Automated Tests: • Unit tests for password hashing and verification • Tests to confirm HTTPS enforced on login and password change endpoints Manual Review: • Inspect database to confirm no plaintext passwords Penetration Testing: • Validate resistance to timing attacks and brute-force attempts Compliance Audit: • Confirm bcrypt configuration and work factor ≥ 10
Notes:		Consider adaptive hashing (bcrypt, Argon2) for future-proofing. Implement rate limiting and account lockout for brute-force protection. Ensure secure password reset flow (tokens expire, HTTPS enforced). Regularly review work factor to align with industry standards. Avoid logging passwords or sensitive data in any system logs

ID#	NFR-SE C-005	JWT Token Security
Scenario:	As a security administrator I want authentication tokens to be secure and properly validated So that user sessions cannot be hijacked or tampered with	
Requirement:	Authentication tokens shall be secure and properly validated.	
Constraints:	Security Constraint: Secret key must never be hardcoded or exposed in logs Compliance Constraint: Must adhere to organizational security policies Performance Constraint: Token validation should not degrade API performance	
Verification Method:	Automated Tests: • Unit tests for token generation and validation • Tests for expired token rejection • Tests for tampered token rejection Manual Review: • Inspect environment configuration for secure key storage Penetration Testing: • Validate resistance to token forgery and replay attacks Compliance Audit: • Confirm adherence to JWT best practices	
Notes:	Consider rotating secret keys periodically for enhanced security. Implement shorter token lifetimes for high-risk operations.	

ID#	<i>NFR-SE C-005</i>	JWT Token Security
		Use refresh tokens for long-lived sessions with secure handling. Ensure HTTPS enforced for all token transmission. Avoid storing sensitive data in JWT payload (keep claims minimal).

ID#	NFR-SE C-006	Role-Based Access Control
Scenario:	<i>As a system administrator</i> <i>I want system operations to be restricted based on user roles</i> <i>So that users only access features and data they are authorized for</i>	
Requirement:	System operations shall be restricted based on user roles.	
Constraints:	Security Constraint: Role validation must occur server-side (frontend hiding is not sufficient) Compliance Constraint: Must adhere to organizational security policies Performance Constraint: RBAC checks should not degrade API performance	
Verification Method:	Automated Tests: • Unit tests for middleware role checks • Integration tests for API responses based on roles Manual Review: • Verify UI elements hidden for unauthorized roles Penetration Testing:	

ID#	<i>NFR-SE C-006</i>	Role-Based Access Control
		- Attempt privilege escalation and bypass attacks Compliance Audit: - Confirm RBAC implementation aligns with policy
	Notes:	Maintain a centralized role definition to avoid inconsistencies. Consider role hierarchy and future extensibility (e.g., adding new roles). Ensure JWT claims cannot be tampered with (signature verification). Implement least privilege principle for all roles. Document role permissions matrix for clarity

ID#	<i>NFR-SE C-007</i>	Session Management
	Scenario:	As a security administrator I want user sessions to be managed securely with automatic expiration So that unauthorized access is minimized and session integrity is maintained
	Requirement:	User sessions shall be managed securely with automatic expiration.
	Constraints:	Security Constraint: Tokens must not be stored in insecure locations (e.g., localStorage without encryption) Compliance Constraint: Must adhere to organizational security policies Performance Constraint: Session checks should not degrade API performance
	Verification Method:	Automated Tests: Unit tests for token expiration handling

ID#	NFR-SE C-007	Session Management
		- Integration tests for logout and redirect behavior Manual Review: - Inspect client-side storage for token clearance Penetration Testing: - Validate resistance to session hijacking and replay attacks Compliance Audit: - Confirm adherence to session management best practices
	Notes:	Consider shorter token lifetimes for high-risk operations. Implement refresh token mechanism for extended sessions. Ensure HTTPS enforced for all token transmission. Avoid storing tokens in localStorage; prefer secure cookies or encrypted storage. Document session timeout policy for clarity.

4.2.3 DATA PROTECTION

ID#	NFR-SE C-008	Input Validation
	Scenario:	As a security administrator I want all user inputs to be validated So that injection attacks and malicious data cannot compromise the system
	Requirement:	All user inputs shall be validated to prevent injection attacks.

ID#	NFR-SE C-008	Input Validation
	Constraints:	Security Constraint: Validation must occur server-side regardless of client-side checks Compliance Constraint: Must adhere to organizational security policies Performance Constraint: Validation should not significantly degrade request handling
	Verification Method:	Automated Tests: • Unit tests for Pydantic schema validation • Integration tests for SQL injection and XSS prevention Manual Review: • Inspect database schema for constraints Penetration Testing: • Attempt SQL injection, XSS, and command injection attacks Compliance Audit: • Confirm adherence to secure coding standards
	Notes:	Use <i>centralized validation logic</i> to avoid duplication. Implement <i>input sanitization</i> and <i>output encoding</i> for XSS prevention. Avoid using <i>eval()</i> or direct shell commands in code. Consider <i>security libraries</i> (e.g., OWASP recommendations). Document <i>validation rules</i> for each input field.

ID#	<i>NFR-SE C-009</i>	HTTPS/TLS Encryption
Scenario:	As a security administrator I want all production data transmission to be encrypted So that sensitive information is protected during transit	
Requirement:	All production data transmission shall be encrypted.	
Constraints:	Security Constraint: No plaintext transmission in production Compliance Constraint: Must adhere to organizational security policies and industry standards Performance Constraint: TLS configuration should not significantly degrade response times	
Verification Method:	Automated Tests: - Integration tests to confirm HTTPS enforced on all endpoints Manual Review: - Inspect SSL/TLS certificate validity and configuration External Audit: - Run SSL Labs test and confirm grade A Penetration Testing: - Validate no downgrade attacks or insecure cipher suites	
Notes:	Use HSTS (HTTP Strict Transport Security) to enforce HTTPS. Configure secure cipher suites and disable weak protocols (e.g., SSL, TLS 1.0/1.1). Automate certificate renewal (e.g., Let's Encrypt).	

ID#	NFR-SE C-009	HTTPS/TLS Encryption
		<i>Monitor for certificate expiration alerts.</i> <i>Document TLS configuration for production environments.</i>

ID#	NFR-SE C-010	Sensitive Data Exposure Prevention
Scenario:	As a security administrator I want to ensure sensitive information is never exposed in API responses or logs So that we prevent data leaks and maintain compliance with security standards	
Requirement:	Sensitive information shall not be exposed in API responses or logs.	
Constraints:	Security Constraint: No sensitive data in API responses or logs Compliance Constraint: Must adhere to organizational security policies and regulations Performance Constraint: Redaction should not significantly impact logging performance	
Verification Method:	Automated Tests: • Unit tests for Pydantic response models excluding sensitive fields • Tests for log redaction functionality Manual Review: • Inspect API responses and logs for sensitive data Penetration Testing:	

ID#	<i>NFR-SE C-010</i>	Sensitive Data Exposure Prevention
		• Attempt to retrieve sensitive data via API or logs Compliance Audit: • Confirm adherence to data protection standards
Notes:		• Implement centralized logging configuration to handle redaction consistently. • Use environment variables for all secrets and credentials. • Consider secret scanning tools (e.g., Git hooks, CI/CD checks) to prevent accidental exposure. • Document sensitive fields list and enforce in code reviews.

4.3 COMPATIBILITY

Compatibility requirements ensure the system works across different platforms, browsers, and environments.

1. *Cross-Browser Compatibility*
2. *Legacy Interface Compatibility*
3. *Etc. Compatibility*

4.3.1 CROSS-BROWSER COMPATIBILITY

ID#	<i>NFR-CO MPAT-00 1</i>	cross-browser compatibility
Scenario:		• <i>Administrators access the Garage Management System using modern desktop web browsers on different operating systems during normal administrative operations. The system is in normal operating condition.</i>
Requirement:		• <i>The system shall function correctly on all supported modern web browsers, ensuring that 100% of core administrative features are fully operational.</i>

ID#	NFR-CO MPAT-00 1	cross-browser compatibility
Constraints:	- Supported browsers include: - Google Chrome version 90 and above - Mozilla Firefox version 88 and above - Microsoft Edge version 90 and above (Chromium-based) - Safari version 14 and above (macOS and iOS) - Older browsers may experience reduced functionality.	
Verification Method:	Execute full regression test cases covering all core administrative functions on each supported browser and compare results for consistency.	
Notes:	Graceful degradation shall be applied on unsupported browsers with appropriate warning messages displayed to users.	

ID#	NFR-CO MPAT-00 2	Responsive Design
Scenario:	Administrators access the system using desktop, laptop, or tablet devices with varying screen resolutions.	
Requirement:	The system interface shall adapt correctly to supported screen resolutions and maintain full functionality.	
Constraints:	- Supported resolutions include: - Desktop: 1920×1080 - Laptop: 1366×768 - Tablet: 768×1024 - Minimum supported width is 768 pixels. - Mobile device support is deferred to a later project phase	
Verification Method:	Test the system UI across supported screen resolutions using responsive testing tools and physical devices.	
Notes:	The system is optimized for administrative use on tablet-sized screens and larger.	

ID#	NFR-CO MPAT-00 3	JavaScript Requirement
Scenario:	A user attempts to access the system with JavaScript enabled or disabled in the browser.	
Requirement:	The system shall require JavaScript (ES6 or higher) to operate and shall display a user-friendly message if JavaScript is disabled.	
Constraints:	- Minimum supported ECMAScript version is ES6 (ES2015). - Polyfills are not required for supported browsers.	
Verification Method:	Disable JavaScript in supported browsers and verify that the system displays an appropriate message.	
Notes:	Frontend code is transpiled using Vite to ensure compatibility with modern browsers.	

4.3.2 LEGACY INTERFACE COMPATIBILITY

ID#	<i>NFR-CO MPAT-00 4</i>	Legacy Data Compatibility
Scenario:	Existing garage data is migrated into the new system.	

ID#	<i>NFR-CO MPAT-00 4</i>	Legacy Data Compatibility
Requirement:	• <i>The system shall support importing legacy data without data loss or corruption.</i>	
Constraints:	• <i>Legacy data formats must be defined prior to migration.</i>	
Verification Method:	• <i>Perform data migration testing and validate record accuracy.</i>	
Notes:	<i>Ensures smooth system transition.</i>	

4.3.3 ETC. COMPATIBILITY

ID#	<i>NFR-CO MPAT-00 5</i>	Device Compatibility
Scenario:	• <i>Administrators access the system from different screen resolutions.</i>	
Requirement:	• <i>The system shall be usable on standard desktop and tablet screen sizes.</i>	
Constraints:	• <i>Mobile optimization is not required for administrators.</i>	
Verification Method:	• <i>Test UI responsiveness across multiple screen sizes.</i>	
Notes:	<i>Improves usability.</i>	

4.4 MAINTAINABILITY

4.4.1 ERROR REPORTING

4.4.1.1 APPLICATION ERRORS

ID#	<i>NFR-MA INT-001</i>	Backend Error Logging
Scenario:	• <i>An application-level error or exception occurs in the backend while processing a user request during normal system operation.</i>	

ID#	NFR-MA INT-001	Backend Error Logging
Requirement:	● <i>The system shall log all backend application errors with sufficient detail to support debugging and issue reproduction.</i>	
Constraints:	● <i>Log entries shall include timestamp (ISO 8601 with timezone), severity level, error type, error message, stack trace, and request context.</i> ● <i>Logs shall be stored in structured JSON format.</i> ● <i>Logs shall be rotated daily and retained for 30 days.</i>	
Verification Method:	● <i>Trigger controlled backend exceptions and verify that log entries contain all required information and are stored correctly.</i>	
Notes:	<i>Structured logging enables efficient searching and integration with log analysis tools.</i>	

ID#	NFR-MA INT-002	User-Friendly Error Messages
Scenario:	A user encounters an error while interacting with the system, such as validation failure, network issue, or unexpected application error.	
Requirement:	The system shall display clear, user-friendly error messages that explain the issue and guide the user on corrective actions.	
Constraints:	- Error messages shall not expose sensitive technical details. - User-facing error messages shall be displayed in Khmer.	
Verification Method:	Induce various error conditions and verify that appropriate, localized messages are displayed to users.	
Notes:	Generic error messages shall be used for unexpected failures, while validation errors shall be specific.	

4.4.1.2 DATABASE ERRORS

ID#	NFR-MA INT-003	Database Error Handling
Scenario:	● <i>A database-related failure occurs during data access, transaction execution, or query processing.</i>	
Requirement:	● <i>The system shall handle database errors gracefully by catching, logging, and preventing unhandled exceptions from reaching the user.</i>	
Constraints:	● <i>Connection failures shall implement retry logic with a maximum of three attempts.</i> ● <i>Constraint violations shall generate meaningful error messages</i> ● <i>Failed transactions shall be rolled back completely.</i>	
Verification Method:	● <i>Simulate database failures and verify proper logging, retries, and user-friendly error handling.</i>	
Notes:	<i>Technical error details are recorded for developers while users receive simplified messages.</i>	

ID#	NFR-MA INT-004	Data Integrity Error Prevention
Scenario:	● <i>A user attempts to create or modify data that violates business rules or database constraints.</i>	
Requirement:	● <i>The system shall prevent data integrity violations through validation at the frontend, backend, and database levels.</i>	
Constraints:	● <i>Frontend validation provides immediate feedback.</i> ● <i>Backend validation enforces schema and business rules.</i> ● <i>Database constraints enforce referential integrity.</i>	
Verification Method:	● <i>Attempt to submit invalid data and verify that it is rejected at all enforcement layers and not persisted.</i>	
Notes:	<i>Ensures consistency and reliability of system data.</i>	

ID#	<i>NFR-CO MPAT-00 5</i>	Device Compatibility
Scenario:	Administrators access the system from different screen resolutions.	
Requirement:	The system shall be usable on standard desktop and tablet screen sizes.	
Constraints:	Mobile optimization is not required for administrators.	
Verification Method:	Test UI responsiveness across multiple screen sizes.	
Notes:	Improves usability.	

ID#	<i>NFR-CO MPAT-00 5</i>	Device Compatibility
Scenario:	Administrators access the system from different screen resolutions.	
Requirement:	The system shall be usable on standard desktop and tablet screen sizes.	
Constraints:	Mobile optimization is not required for administrators.	
Verification Method:	Test UI responsiveness across multiple screen sizes.	
Notes:	Improves usability.	

4.4.1.3 API ERRORS

ID#	<i>NFR-MA INT-005</i>	Standardized API Error Responses
Scenario:	As a frontend developer I want API errors to follow a consistent format	

ID#	NFR-MA INT-005	Standardized API Error Responses
		So that the client can parse and display error messages uniformly and reliably
Requirement:		API errors shall follow a consistent format for easy client handling.
Constraints:		Consistency Constraint: All endpoints must return errors in the standardized format Security Constraint: No sensitive data in error messages Compliance Constraint: Must adhere to organizational API standards
Verification Method:		Automated Tests: - Unit tests for error response generation - Integration tests for all endpoints returning standardized errors Manual Review: - Inspect sample API responses for compliance Frontend Validation: - Confirm client can parse and display errors correctly
Notes:		<i>Implement a centralized error handling middleware in the backend.</i> <i>Define error codes in a shared constants file for consistency.</i> <i>Avoid exposing internal stack traces or sensitive details in production.</i> <i>Document the error response schema for API consumers.</i>

4.4.1.4 FRONTEND ERRORS

ID#	NFR-MA INT-006	Frontend Error Boundary
Scenario:	As a frontend developer I want React error boundaries to catch and handle component errors So that the application does not crash entirely and provides a user-friendly fallback	
Requirement:	React error boundaries shall catch and handle component errors.	
Constraints:	Consistency Constraint: All major sections must be wrapped in error boundaries Security Constraint: Error messages must not expose sensitive data Performance Constraint: Error boundaries should not impact rendering performances	
Verification Method:	Automated Tests: • Unit tests for error boundary components • Integration tests simulating component errors Manual Review: • Trigger errors and confirm fallback UI appears Frontend Validation: • Check console logs for error details	
Notes:	Use React’s built-in componentDidCatch or ErrorBoundary pattern. Consider remote error logging for production (e.g., Sentry, LogRocket). Ensure fallback UI is accessible and user-friendly. Document which sections are wrapped by error boundaries.	

ID#	NFR-MA INT-007	Network Error Handling
Scenario:	As a user I want the frontend to handle network failures gracefully So that I understand what's happening and can take appropriate action without frustration	
Requirement:	Frontend shall handle network failures gracefully.	
Constraints:	Consistency Constraint: All API calls must use the same error-handling strategy Performance Constraint: Retry logic should not overload the server Security Constraint: No sensitive data exposed in error messages	
Verification Method:	Automated Tests: • Simulate API unreachable scenario and verify offline message • Test retry logic with exponential backoff Manual Review: • Check loading indicators on slow network User Testing: • Validate clarity of error messages and user experience	
Notes:	Consider using React Query or Axios interceptors for centralized error handling. Implement progressive web app (PWA) features for offline support. Log network failures for monitoring and analytics.	

ID#	<i>NFR-MA</i> <i>INT-007</i>	Network Error Handling
		Ensure accessibility compliance for error messages and indicators

APPENDIX A. **CONSIDERATIONS**

CONSIDERATIONS FOR COTS, MOTS, and CUSTOM IMPLEMENTATION	
COTS	• For COTS products, many of the non-functional requirements may not apply as they are already built into the COTS application and there is little that can be changed. However, external to the application, there are still a number of non-functional requirements that are applicable, which are those involving the environment where the COTS application is hosted. Therefore, non-functional requirements such as performance, availability, some security, and others are applicable and should be specified.
MOTS	• For MOTS, all parts of the application that is not modified, the non-functional requirements are limited, as in COTS. Also, specifying non-functional requirements related to items such as maintainability of code, security of the application, etc., the effect of these non-functional requirements is limited as they would only apply to the modified parts of the application. Therefore, MOTS should follow the approach of COTS when specifying non-functional requirements unless the project determines that there is value in adding non-functional requirements that only apply to part of the software application.
CUSTOM	• For custom developments, all non-functional requirements apply and should be specified consistent with the needs of the stakeholders and the project.